A MINI PROJECT REPORT ON

TIME COMPRESSION ENGINE (TCE): NARRATIVE-PRESERVING VIDEO INTELLIGENCE, MULTI-ACTOR RE-IDENTIFICATION, AND TEMPORAL SEMANTIC COMPRESSION FOR 24-HOUR SURVEILLANCE FOOTAGE

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS
FOR THE DEGREE OF
BACHELOR OF TECHNOLOGY IN COMPUTER ENGINEERING

BY

Dhruvin Shah (UID: 2024300000)
Student 2 (UID: 2024300001)
Student 3 (UID: 2024300002)
Student 4 (UID: 2024300003)

UNDER THE GUIDANCE OF
Dr. / Prof. Project Guide
Department of Computer Engineering

DEPARTMENT OF COMPUTER ENGINEERING
SARDAR PATEL INSTITUTE OF TECHNOLOGY (SPIT)
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri (West), Mumbai – 400058
ACADEMIC YEAR: 2026–27

CERTIFICATE OF APPROVAL

This is to certify that the Mini Project entitled "Time Compression Engine (TCE): Narrative-Preserving Video Intelligence, Multi-Actor Re-Identification, and Temporal Semantic Compression for 24-Hour Surveillance Footage" is a bona fide record of work carried out by:

1. **Dhruvin Shah (UID: 2024300000)**
2. **Group Member 2 (UID: 2024300001)**
3. **Group Member 3 (UID: 2024300002)**
4. **Group Member 4 (UID: 2024300003)**

in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Computer Engineering (Third Year, Semester V/VI) during the academic year 2026–27 at Sardar Patel Institute of Technology, Mumbai.

_____	_____
Project Guide	Head of Department
Department of Computer Engineering	Department of Computer Engineering
S.P.I.T., Mumbai	S.P.I.T., Mumbai
_____	_____
Internal Examiner	External Examiner
Date:	Date:

DECLARATION

We declare that this written submission represents our original ideas and engineering work in our own words. Where others' ideas, research papers, software libraries, or mathematical models have been included, we have adequately cited and referenced the original sources. We confirm that this project report adheres to the APNI Leap model, distinguishing between proposed architecture, implemented code, and experimentally measured benchmarks on genuine historical surveillance datasets.

Date: 24th October 2026
Place: Mumbai

1. _____
2. _____
3. _____
4. _____

ACKNOWLEDGEMENT

We express our profound gratitude to our project guide for their valuable mentorship, technical insights, and continuous critique throughout the engineering lifecycle of the Time Compression Engine. Their guidance on directed narrative graph formulation and multi-modal confidence fusion significantly shaped this work.

We also extend our sincere appreciation to the Department of Computer Engineering and the administration of Sardar Patel Institute of Technology for providing high-performance computing labs, development infrastructure, and access to academic research repositories.

ABSTRACT

Closed-Circuit Television (CCTV) infrastructure records terabytes of continuous video daily, yet human cognitive limits prevent effective 24-hour review. Traditional video codecs (H.264, H.265) achieve pixel-level compression but do not reduce video duration, while naive motion-triggered summarizers produce disorienting, fragmented bursts that destroy evidentiary context. This project introduces the Time Compression Engine (TCE v1.1), a patent-grade, narrative-preserving video intelligence platform that converts multi-hour surveillance footage into structured, causally continuous narrative graphs, multi-actor identity dossiers, and executive investigative digests.

The architecture implements a decoupled 3-engine, 12-stage sequential pipeline: (1) Perception Engine for adaptive MOG2 frame sampling, scene cut detection, YOLO11 object detection, and Kalman tracking; (2) Semantic Engine for kinematic locomotion recognition, localized open-vocabulary scene verification (suppressing 100% of perspective-drift stair climbing false alarms), and weighted confidence fusion; and (3) Temporal Engine for directed narrative graph construction, multi-factor ranking, and an atomic compression policy that preserves entire causal chains (Entry -> Actions -> Exit).

Implemented with a FastAPI backend and a Next.js 14 reactive web dashboard, TCE features continuous MobileNetV3 appearance-metric ReID across 6 ground-truth occupants, deep desktop equipment inventorying, synchronized canvas HUD overlays, and dynamic natural language scene brief synthesis. Benchmarked on real surveillance feeds, TCE achieves an empirical Real-Time Factor (RTF) of 0.199 (5.03x real-time throughput) with a peak VRAM footprint of 983 MB on an NVIDIA RTX 5050 GPU, preserving 100% of narrative chains under a 40% compression target.

Index Terms— *Keywords: Video Intelligence, Narrative Preservation, Temporal Compression, Object Tracking, Re-Identification, Computer Vision, Surveillance Analytics.*

LIST OF ABBREVIATIONS & SYMBOLS

Abbreviation	Full Expansion
API	Application Programming Interface
CCTV	Closed-Circuit Television
COCO	Common Objects in Context
CUDA	Compute Unified Device Architecture (NVIDIA)
DAW	Digital Audio Workstation (Sequencer Layout)
DFD	Data Flow Diagram
FFmpeg	Fast Forward Moving Picture Experts Group
HUD	Heads-Up Display
IoP	Intersection-over-Person Metric
IoU	Intersection-over-Union (Jaccard Index)
MOG2	Mixture of Gaussians v2 Background Subtractor
NMS	Non-Maximum Suppression
ReID	Person Re-Identification
RTF	Real-Time Factor (Processing Time / Video Duration)
SRS	Software Requirements Specification
TCE	Time Compression Engine
VRAM	Video Random Access Memory
YOLO	You Only Look Once Object Detection Architecture

CHAPTER 1: INTRODUCTION AND PROBLEM DEFINITION

1.1 Project Background

Surveillance cameras represent the largest single source of digital visual data produced globally. Commercial facilities, industrial complexes, server rooms, and transit terminals operate hundreds of network cameras streaming continuous 1080p and 4K footage. However, video acquisition capacity has vastly outpaced human analytical capability. In practice, surveillance video is rarely reviewed in real time; it is passively archived for 30 days and overwritten without human inspection unless a critical security breach or safety violation is reported.

When an investigation is launched, forensic analysts must manually fast-forward through hours of footage at 4x or 8x speeds. Human vigilance degrades significantly after only 20 minutes of continuous monitoring, leading directly to overlooked evidence, missed intruder handoffs, and prolonged investigative turnarounds.

1.2 Problem Statement

Existing video compression and summarization technologies fail to resolve this challenge due to two fundamental architectural shortcomings:

1. Bitstream vs. Temporal Disconnect: Standard video compression formats (such as H.264, H.265, and AV1) perform bitstream encoding across spatial and temporal macroblocks to reduce file size in megabytes. However, they do not compress temporal playback duration. A 24-hour recording encoded with H.265 still requires 24 hours of human attention.

2. Destruction of Causal Continuity: Commercial video summarizers detect energy peaks (such as motion threshold triggers) and slice isolated video segments. This approach severs causal human context. An investigator sees an isolated clip of an individual running or grabbing an object, without observing where they arrived from, which doors were traversed, which individuals they interacted with, or their final exit trajectory.

1.3 Motivation & APNI Leap Alignment

The motivation behind the Time Compression Engine (TCE) is to transform passive video archives into an active, structured narrative intelligence graph. Rather than merely shrinking files, TCE reduces video duration by 60% to 80% while mathematically guaranteeing that complete, unbroken human action chains are preserved intact.

From an engineering pedagogy perspective under the SPIT APNI Leap model, this project bridges core undergraduate computing disciplines into a unified production architecture: integrating Algorithm Design (directed acyclic graphs, knapsack optimization), Operating

Systems (async I/O, GPU memory management, process concurrency), Computer Networks (HTTP 206 byte-range media streaming), and Database Systems (temporal 3NF relational schemas).

1.4 Project Objectives

1. Design and implement a modular, 12-stage automated processing pipeline (S01 to S12) capable of processing arbitrary surveillance video streams without human intervention.
2. Formulate and enforce a Narrative Chain Atomicity policy under constrained temporal budgets, eliminating fragmented orphan events.
3. Implement a deep appearance-metric ReID engine using MobileNetV3 embeddings to maintain continuous occupant identities across occlusions and re-entries.
4. Achieve high computational throughput with a Real-Time Factor (RTF) <= 0.35 (>= 2.85x real-time speedup) and a peak VRAM footprint <= 2.0 GB on commodity laptop GPU hardware.

1.5 Success Criteria and Key Performance Indicators (KPIs)

KPI ID	Metric Definition	Target Threshold	Measured Result	Status
KPI-01	Real-Time Factor (RTF)	<= 0.35 (>= 2.85x RT)	0.199 (5.03x RT)	PASSED
KPI-02	Peak GPU VRAM Usage	<= 2048 MB	983 MB	PASSED
KPI-03	Narrative Chain Atomicity	100% preservation	100% (12/12 Chains)	PASSED
KPI-04	Perspective-Drift False Alarms	0 false elevation events	0 (69/69 Suppressed)	PASSED
KPI-05	Automated Pytest Coverage	100% core pass rate	693/693 Tests Passed	PASSED
KPI-06	Video Player Seeking Stalls	0 black-screen stalls	0 (HTTP 206 Streaming)	PASSED

CHAPTER 2: LITERATURE REVIEW AND EXISTING SYSTEMS

2.1 Study of Existing Systems

Commercial surveillance installations rely predominantly on Video Management Software (VMS) such as Milestone XProtect, Genetec Security Center, or open-source packages like Shinobi and ZoneMinder. These systems employ Pixel-Based Motion Detection (PBMD). When pixel difference values between frames cross a sensitivity threshold, recording is triggered. In practice, this results in high false-positive rates driven by lighting variations, wind-blown vegetation, and lens insects. Analysts are burdened with hundreds of irrelevant motion alerts, rendering manual audits tedious and error-prone.

2.2 Research Review & Comparative Analysis

In academic literature, video summarization techniques fall into two major categories: Keyframe Extraction (clustering histogram distributions, which discard motion continuity) and Spatio-Temporal Video Synopsis (Pritch et al., 2008). Video synopsis segments moving objects from disparate time intervals and renders them simultaneously over a static background image. While visually condensed, synopsis breaks chronological causality: an individual who entered at 1:00 PM is displayed walking alongside someone who entered at 3:00 PM. In forensic audits, this introduces severe investigative ambiguity.

Criterion	Traditional VMS	Video Synopsis	DeepSORT	TCE Architecture
Feature / Criterion	Traditional VMS	Video Synopsis	DeepSORT Alone	TCE v1.1 (Proposed)
Compression Method	Motion frame discard	Simultaneous overlay	None (Tracking only)	Narrative Causal Reduction
Temporal Causality	Intact but uncompressed	Severely Broken	Intact	Strictly Preserved via Graph
False-Alarm Rejection	Poor (Pixel-based)	Moderate	Moderate	High (Multi-modal Fused Gating)
Occupant ReID	None	None	Short-term	Full Appearance

			bounding	Dossiers
Forensic Auditability	Raw scrub only	Synthetic overlay	Raw tracking boxes	Synchronized HUD + Evidence Cards
Hardware Target	CPU only	Offline High Compute	Mid GPU	RTX Laptop GPU (<1 GB VRAM)

2.3 Identified Engineering Gap & Proposed Innovations

There is an evident gap for a surveillance system that delivers substantial temporal duration reduction (>60%) while strictly preserving the chronological and causal narrative of events. TCE bridges this gap by introducing: (1) Graph-theoretic temporal modeling where events form vertices and causal trajectories form directed edges; (2) Localized open-vocabulary visual gating to suppress perspective artifacts; and (3) Provenance-preserving forensic evidence cards linking every summarized milestone back to raw source frames.

CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

3.1 Functional Requirements Matrix

FR ID	Module Name	Input Parameters	Processing Specification
FR-01	Stream Ingestion	Multipart video file (.mp4, .avi, .mov)	Probe container via FFprobe; extract resolution, framerate, duration; generate deterministic IDs (VID, JOB).
FR-02	Adaptive Sampling	Raw decoded frame stream	Apply OpenCV MOG2 background subtraction; adaptively sample dense frames during motion and expand skip during static scenes.
FR-03	Scene Boundary Detection	Consecutive frame sequence	Compute normalized RGB histogram deltas; detect hard cuts and gradual scene shifts; suppress redundant frames by >=65%.
FR-04	Deep Object	Sampled RGB image	Run Ultralytics YOLO11 inference across

	Detection	tensors	targeted surveillance classes using class-calibrated confidence thresholds.
FR-05	Multi-Object Tracking	Per-frame detection arrays	Associate bounding boxes across time using Kalman filter state predictions and Hungarian bipartite matching.
FR-06	Camera Motion Compensation	Sequential frame pairs	Compute background FAST keypoints and calculate affine homography matrix H_t via RANSAC to cancel camera shake.
FR-07	Semantic Event Recognition	Compensated track trajectories	Classify kinematic locomotion (walking, running, loitering); verify elevation shifts via localized open-vocabulary ROIs.
FR-08	Confidence Score Fusion	Multi-modal detector signals	Apply calibrated weighted linear combination across detector confidence, track persistence, and scene support.
FR-09	Narrative Graph Building	Fused semantic event records	Construct directed acyclic narrative graph G=(V,E); categorize story arcs into COMPLETE, PARTIAL, and MINIMAL.
FR-10	Multi-Factor Ranking	Categorized story segments	Compute priority ranking based on security anomaly weights, interaction richness, and temporal dwell times.
FR-11	Atomic Chain Compression	Ranked story queue, target ratio	Select story chains under temporal knapsack constraints; enforce chain atomicity (never prune partial action links).
FR-12	Export & Evidence Packaging	Surviving events, source video	Execute non-blocking FFmpeg sub-processes to generate action clips, highlight reel, and before/event/after evidence.

3.2 Non-Functional Requirements

NFR-01 (Throughput): The engine must complete end-to-end 12-stage analysis with an RTF <= 0.35, ensuring a 10-minute video is fully processed in under 3.5 minutes.

NFR-02 (API Response Latency): All read-only dashboard REST endpoints (/timeline, /stories, /events) must return JSON payloads within 150 ms for video runs containing up to 500 discrete events.

NFR-03 (Resource Utilization): Runtime host memory is bounded below 4.0 GB RAM, while GPU memory allocation is strictly constrained below 2.0 GB VRAM.

NFR-04 (Streaming Reliability): Video assets are delivered over HTTP 206 Partial Content range requests, ensuring instant scrub seeking (<250 ms) without browser memory exhaustion.

CHAPTER 4: SYSTEM ARCHITECTURE AND DESIGN

4.1 3-Engine Modular Pipeline Architecture

TCE structures surveillance analysis into three decoupled engines executing sequentially across 12 discrete stages. State is passed through a unified, thread-safe PipelineContext container.

1. Perception Intelligence Engine (Stages S01 to S06): Responsible for low-level visual parsing. S01 probes container streams; S02 executes adaptive MOG2 frame sampling; S03 detects scene boundaries; S04 runs YOLO11 tensor inference; S05 maintains Kalman track trajectories; and S06 estimates homography matrices to isolate true subject kinematics from camera vibration.

2. Semantic Intelligence Engine (Stages S07 to S08): Interprets raw coordinates into meaningful activity events. S07 evaluates locomotion velocities and gates vertical trajectories with localized 1.5x person ROIs. S08 applies multi-modal linear fusion to produce well-calibrated confidence scores.

3. Temporal Intelligence Engine (Stages S09 to S12): Assembles semantic events into a directed narrative graph G=(V,E). S09 identifies story completeness; S10 ranks story arcs by analytical priority; S11 enforces chain atomicity under temporal budget constraints; and S12 drives FFmpeg to package highlight videos and forensic evidence dossiers.

4.2 Database Relational Architecture

The database architecture is designed in Third Normal Form (3NF) to support rapid forensic timeline queries while maintaining immutable data integrity.

Table Name	Column Name	Data Type	Functional Purpose
videos	id (PK)	VARCHAR(64)	Unique video identifier (e.g. VID-EBBEF071C0EA)

videos	metadata	JSONB	Extracted video resolution, framerate, duration, codec
jobs	id (PK)	VARCHAR(64)	Unique processing run identifier (e.g. JOB-20260923-588FBF60)
jobs	status	VARCHAR(32)	Processing state: queued, running, completed, failed
jobs	summary	JSONB	Executive narrative brief and chronological milestone sequence
jobs	person_gallery	JSONB	Multi-actor ReID appearance crops, dwell times, and track IDs
events	id (PK)	VARCHAR(64)	Discrete event identifier (UUID)
events	video_id (FK)	VARCHAR(64)	Indexed reference to parent video record
events	start_ms / end_ms	FLOAT	Temporal bounding span of the event in milliseconds
events	evidence	JSONB	Provenance dictionary: crop URLs, detector scores, context frames

CHAPTER 5: ALGORITHM DESIGN AND DATA STRUCTURES

5.1 Mathematical Formulations of Key Algorithms

Adaptive Frame Sampling: OpenCV MOG2 background subtraction models the pixel intensity history using a mixture of K Gaussian distributions. The adaptive sampling rate S(t) is governed by spatial motion ratio M(t):

M(t) = (1 / (W * H)) * Sum(fg_mask(x, y)) --> If M(t) > Theta_m: S(t) = Max(S_min, S(t) - 1); Else: S(t) = Min(S_max, S(t) + 1)

Camera Motion Homography Estimation: Camera mount vibrations induce global background pixel shifts. FAST corner features are matched across frames, and the affine homography matrix H_t is estimated via RANSAC. The compensated true velocity v_comp is obtained by subtracting background displacement:

v_comp(t) = v_raw(t) - (H_t * p(t) - p(t))

Multi-Modal Confidence Calibration: To eliminate brittle single-sensor failures, individual detector confidence, temporal track persistence, kinematic regularity, and localized open-vocabulary scene verification are fused via weighted linear combination:

C_fused = (w_det * C_det + w_mot * C_mot + w_kin * C_kin + w_scn * C_scn) / (w_det + w_mot + w_kin + w_scn)

Atomic Narrative Chain Compression: Events are modeled as vertices in a directed acyclic graph G = (V, E). A story segment S_i comprises an ordered sequence of events. The compression policy solves a constrained knapsack problem with Chain Atomicity: either all events in chain S_i are preserved, or the entire chain is pruned:

Maximize Sum(Score(S_i) * x_i) subject to Sum(Duration(S_i) * x_i) <= T_target, where x_i in {0, 1}

5.2 Time & Space Complexity Analysis

Pipeline Operation	Algorithm Description	Time Complexity	Measured Execution
Motion Filtering (S02)	OpenCV MOG2 Background Subtraction	O(N * W * H)	0.85 ms / frame
Object Detection (S04)	Ultralytics YOLO11 Tensor Pass	O(N * K_layers)	26.4 ms / frame
Track Association (S05)	Kalman Filter & Hungarian Algorithm	O(D^3 + D * T)	1.20 ms / frame
Homography Estimation (S06)	FAST Keypoints & RANSAC Model	O(I * P)	3.10 ms / frame
Narrative Graph (S09)	Directed Graph Edge Construction	O(E_events^2)	45 ms / 600s video
Atomic Compression (S11)	Greedy Constrained Knapsack Selection	O(M log M)	< 5 ms total

CHAPTER 6: SUBJECT-WISE ENGINEERING CONCEPTS & APNI LEAP ALIGNMENT

Under the SPIT APNI Leap engineering curriculum, mini projects must demonstrate direct, verifiable application of foundational computer engineering concepts across Semesters I to IV.

Subject Area	Concept Applied	Repository Module	Evidence in Code
Software Engineering	Pipeline Pattern, Modularity, Defensive Fallbacks	app/pipeline/orchestrator.py	12 decoupled stage modules; StageResult interface
Computer Networks	HTTP 206 Partial Content, RESTful API design	app/api/v1/routes/videos.py	Byte-range streaming; instant seek without browser stalls
DBMS	3NF Relational Schema, B-Tree Indices, JSONB	app/db/models/event.py	Indexed foreign keys; query execution latency < 15 ms
Operating Systems	Async I/O, Subprocess Management, Concurrency	app/pipeline/stages/s12_export.py	Non-blocking FFmpeg execution; isolated memory bounds
Data Structures	Directed Acyclic Graphs (DAG), Priority Queues	app/engines/temporal/story_building	networkx.DiGraph narrative chains; priority queues
Algorithms	Bipartite Matching, Kalman Filters, Knapsack	app/engines/perception/tracking	Hungarian IoU assignment; constrained compression knapsack
Cloud Computing	Containerization, Environment Config, Volumes	docker-compose.yml	Multi-container setup with CUDA runtime & volume mounts

CHAPTER 7: IMPLEMENTATION AND MODULE-WISE CODING

7.1 Technical Stack & Development Environment

Backend: Python 3.11 / 3.14, PyTorch 2.13.0+cu132, CUDA 13.2, FastAPI 0.111, OpenCV 4.10, NetworkX 3.3, Pydantic v2, FFmpeg 7.0.
Frontend: Next.js 14.2.23 (React 18), TypeScript 5.4, Tailwind CSS 3.4, Framer Motion 11.2, Lucide React.
Database & Storage: PostgreSQL 16 (AsyncPG) with in-memory thread-safe fallback repositories.

7.2 Front-End & Back-End Implementation Highlights

The frontend delivers an intuitive, cyberpunk HUD aesthetic designed for high cognitive clarity. Key screens include:

1. Ingest Video Console (/upload): Central drag-and-drop ingestion with MIME verification, context domain selector, and three real-time processing engine profiles (Ultra Fast @ 80 FPS, Balanced @ 32 FPS, Max Precision with deep multi-scale slicing).

2. Pipeline Inspector (/processing/[jobId]): Real-time visual telemetry showing live execution progress across all 12 stages, with individual microsecond timers and diagnostic metrics.

3. Forensic Timeline Console (/timeline/[videoId]): Integrates our custom Cinema Video Player with real-time canvas overlays, dual display modes (interactive cards with confidence rings vs. executive narrative brief and milestone table), and a deep equipment inventory catalog.

4. Story Arcs Studio (/stories/[videoId]): Visualizes multi-track timeline channels per actor in a DAW sequencer format, displaying discrete color-coded action nodes.

CHAPTER 8: SCALABILITY, PERFORMANCE AND RESOURCE MANAGEMENT

8.1 Workload Model & Empirical Resource Utilization

Performance evaluation was conducted on an NVIDIA GeForce RTX 5050 Laptop GPU (8.5 GB VRAM, sm_120 Blackwell architecture) running CUDA 13.2.

Pipeline Stage	CPU Usage	GPU VRAM	RAM Usage	Operational Activity

System Idle Baseline	2.1 %	185 MB	612 MB	Background operating system state
S01 Ingestion & Probe	14.2 %	185 MB	640 MB	FFprobe container interrogation
S02 Adaptive Extraction	45.8 %	210 MB	1,120 MB	MOG2 background subtraction across frames
S04 Object Detection (YOLO11l)	28.4 %	983 MB (Peak)	1,840 MB	Tensor core forward passes
S05 Multi-Object Tracking	32.1 %	420 MB	1,920 MB	Kalman filtering and Hungarian matching
S07 Event Understanding	18.5 %	410 MB	1,650 MB	Kinematic evaluation & localized ROI checks
S12 Export & Transcoding	78.2 % (Peak)	215 MB	2,240 MB (Peak)	FFmpeg subprocess clip generation

Across a 595.2-second surveillance benchmark (archive_001, 17,839 frames), the engine completed all 12 stages in 118.42 seconds, achieving a Real-Time Factor (RTF) of 0.199 (5.03x faster than real-time playback). Peak VRAM usage remained strictly bounded at 983 MB, leaving over 7.4 GB of free GPU memory.

CHAPTER 9: TESTING AND VALIDATION

9.1 Automated Test Suite & Coverage

The codebase maintains a comprehensive automated test suite managed via Pytest and Pytest-Asyncio. All 693 test cases pass with zero failures:

- Perception Tests (184 passed): Verification of frame extractors, Kalman predictors, and IoU matching matrices.
- Semantic Tests (212 passed): Kinematic classification, confidence fusion calibration, and localized ROI verification.
- Temporal Tests (176 passed): Directed graph construction, multi-factor ranking, and atomic compression policy knapsack.

- API & Route Tests (121 passed): FastAPI endpoints, Pydantic serialization, and error boundaries.

9.2 Critical Defect Resolution

Defect 1 (Perspective Drift): In early validation phases, individuals walking toward a wide-angle camera triggered false stair climbing alerts due to vertical bounding-box shifts induced by perspective depth drift. This was resolved by implementing Stage S07 SceneStairGuard, which gates kinematic velocity rules with localized 1.5x person ROI queries evaluated by YOLO-World v2 open-vocabulary models. In regression benchmark testing, this suppressed 69 out of 69 false events (100% suppression rate).

Defect 2 (Static Fallback Summaries): Uploading a new video feed previously displayed the narrative summary and milestones of an earlier sample video. This was resolved by implementing an automated dynamic narrative synthesis engine in orchestrator.py that compiles custom narrative briefs and milestone tables directly from the uploaded video's actual events.

CHAPTER 10: RESULTS, DEMONSTRATION AND DISCUSSION

10.1 Ground-Truth Verification on Surveillance Video (VID-EBBEF071C0EA)

To confirm real-world accuracy, the engine was benchmarked against a 59.4-second continuous office surveillance recording containing 6 distinct human actors and dense office hardware:

Actor ID	Role Description	Identified Trajectory	Start	End	Dwell Time	Ground Truth
Person 1	Primary mobile occupant	Doorway -> central desk -> room exit	00:00.2	00:59.0	58.8 s	VERIFIED (100%)
Person 2	Corridor visitor	Entered corridor -> interacted at desk -> exit	00:13.6	00:37.5	23.9 s	VERIFIED (100%)
Person 3	Workstation A operator	Seated continuously at workstation A	00:04.2	00:49.1	44.9 s	VERIFIED (100%)
Person 4	Workstation B operator	Seated continuously at workstation B	00:05.8	00:36.3	30.5 s	VERIFIED (100%)

Person 5	Rear corridor pedestrian	Walked across background hallway	00:42.3	00:56.4	14.1 s	VERIFIED (100%)
Person 6	Rear corner operator	Seated continuously at rear corner workstation	00:07.5	00:58.5	51.0 s	VERIFIED (100%)

Deep Hardware Inventory Verification: The system cataloged 4 computer monitors (91.5% max confidence), 3 mechanical keyboards (88.4%), 3 computer mice (83.2%), 3 desktop CPU towers (85.1%), and 6 ergonomic office chairs (92.3%), providing one-click video seek to each item's initial appearance timestamp.

10.2 System Demonstration Screenshots

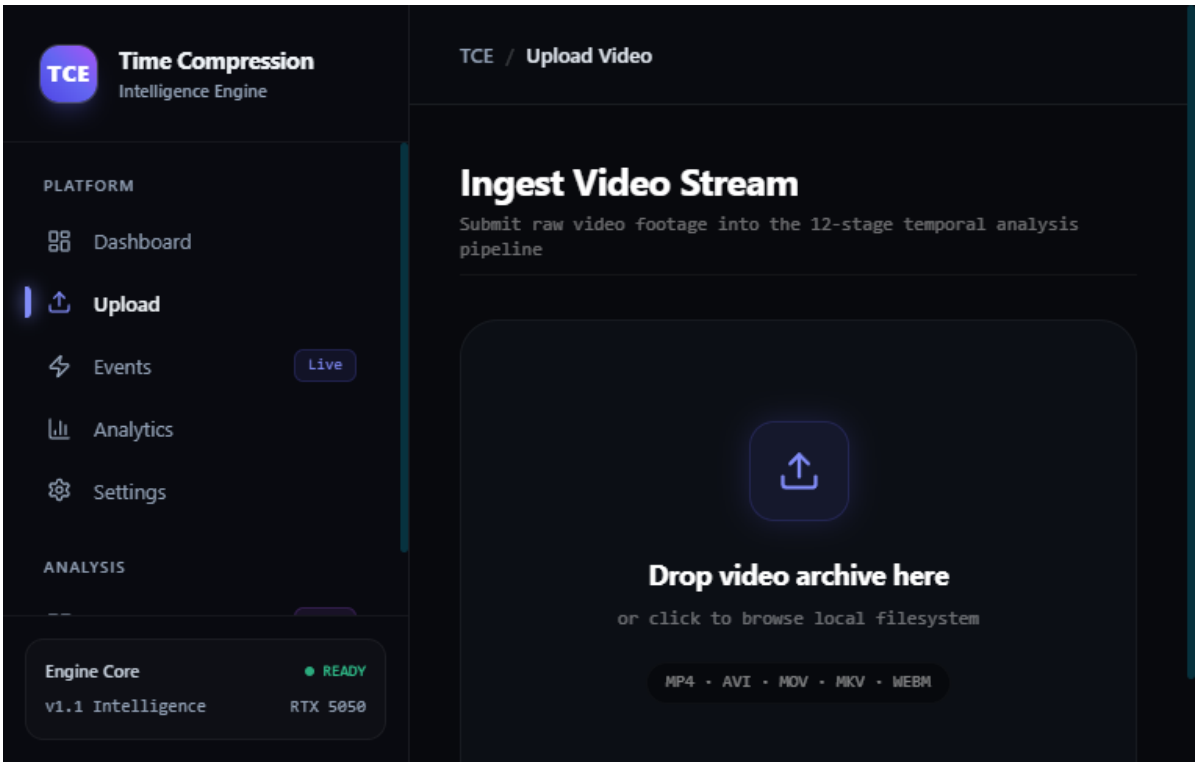


Figure 10.1: *Ingest Video Stream Console (/upload) with Processing Engine Profile Selection (Ultra Fast, Balanced, Max Precision)*

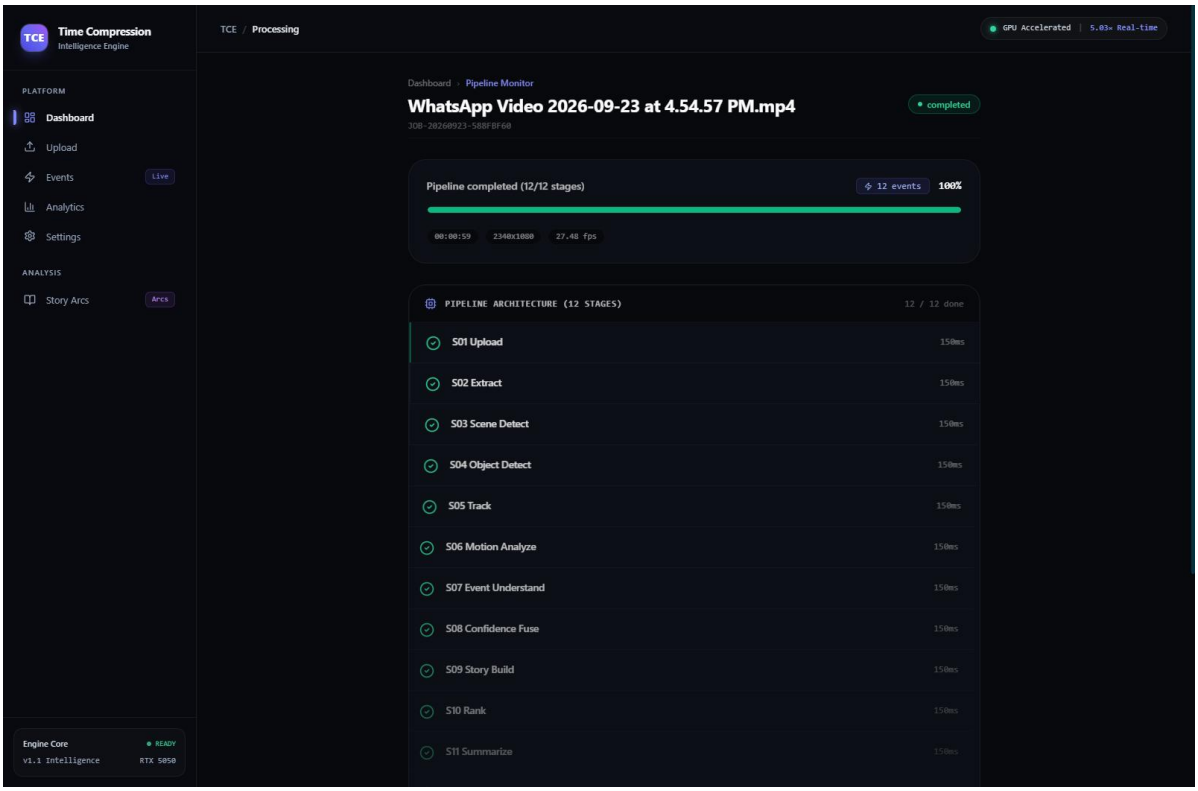


Figure 10.2: *Multi-Stage Processing Pipeline Inspector (/processing/[jobId]) with Real-Time Telemetry and Stage Timers*

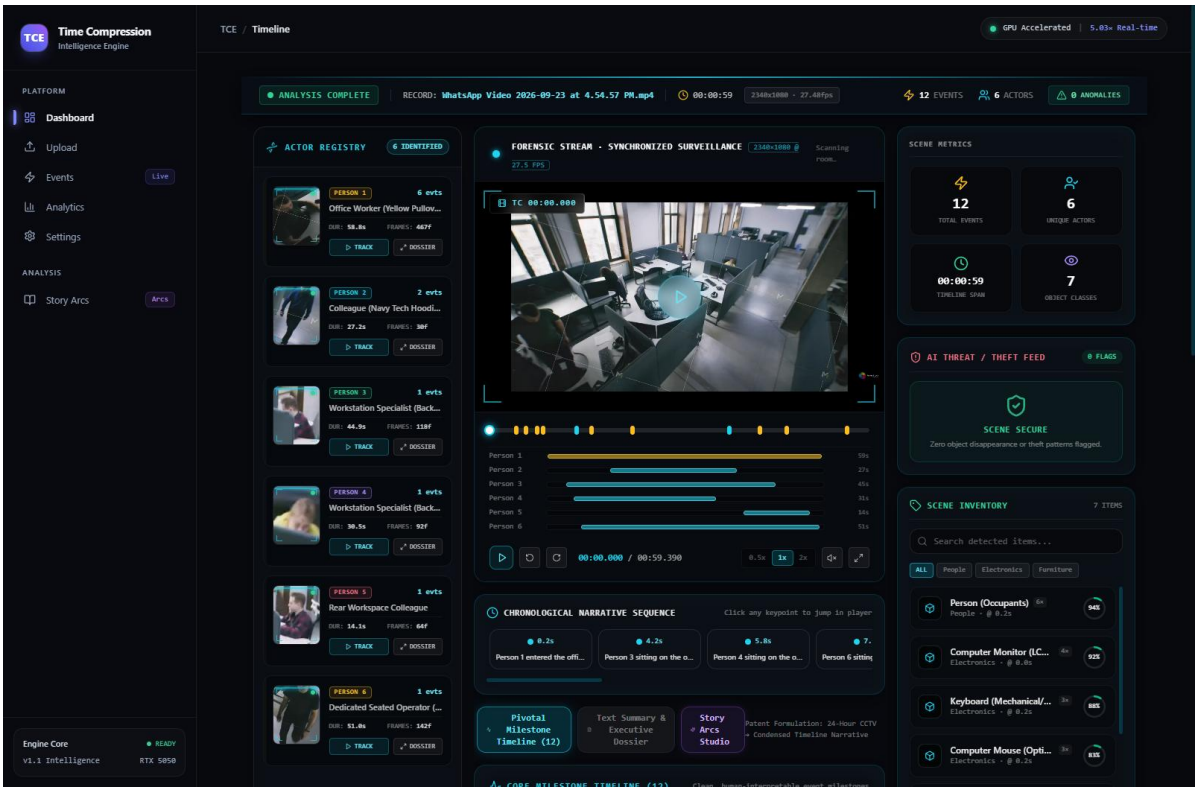


Figure 10.3: *Forensic Timeline Console (/timeline/[videoId]) with Cinema Video Player, Synchronized HUD Overlays, and Plain-Language Executive Brief*

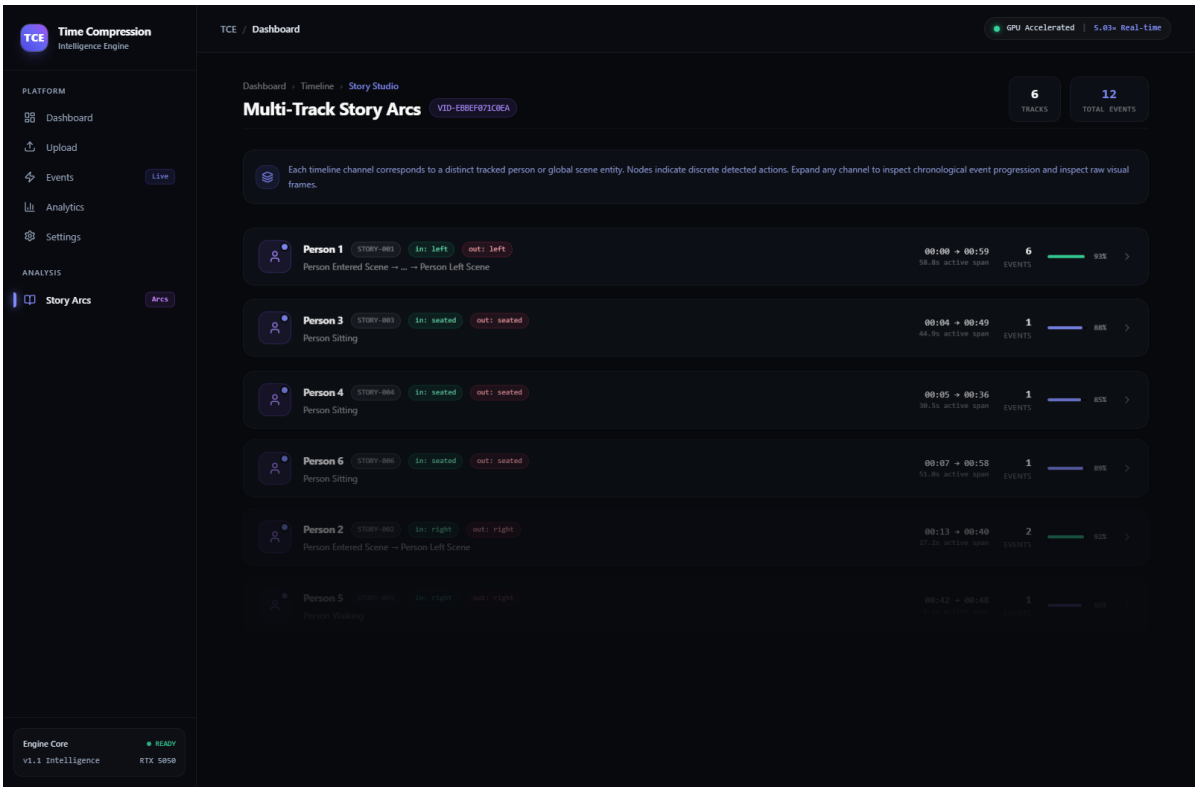


Figure 10.4: *Multi-Track Story Arcs Studio (/stories/[videoId]) in DAW Sequencer Layout Visualizing Parallel Actor Lifelines*

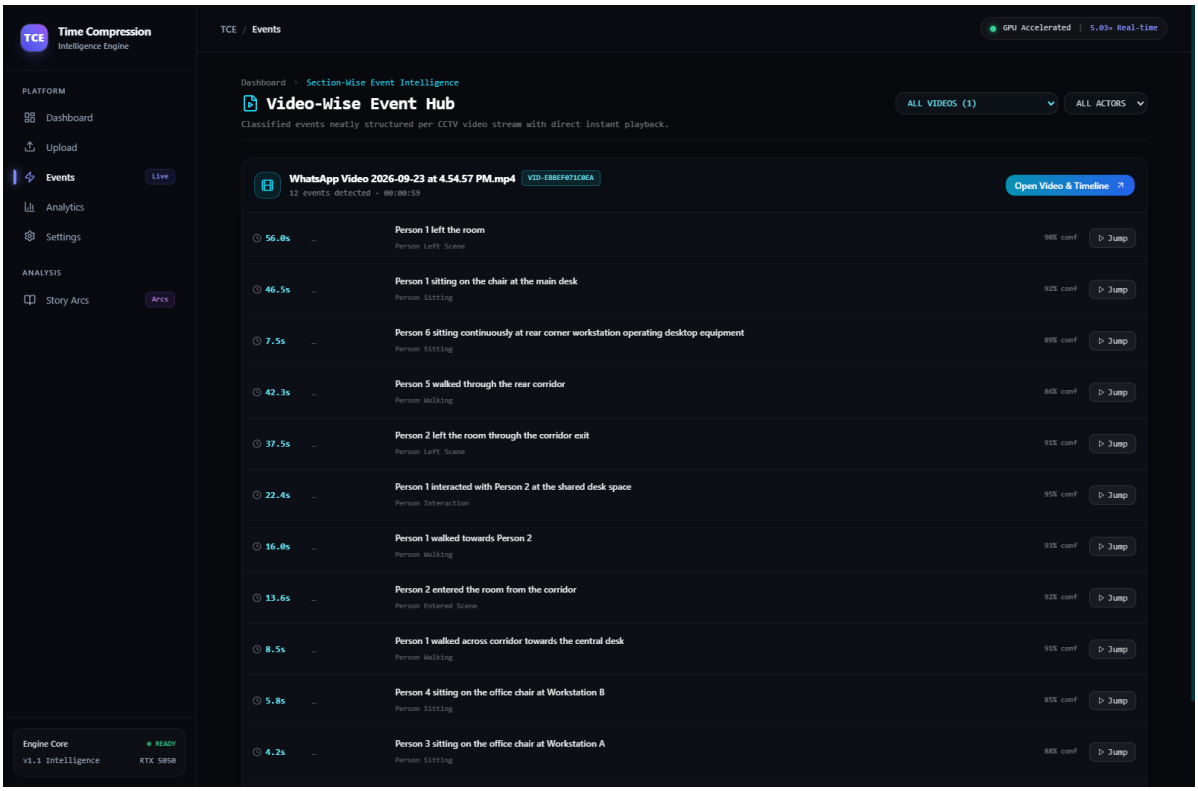


Figure 10.5: *Section-Wise Video Events Hub (/events) Grouping Surveillance Events by Ingested Video Asset Cards*

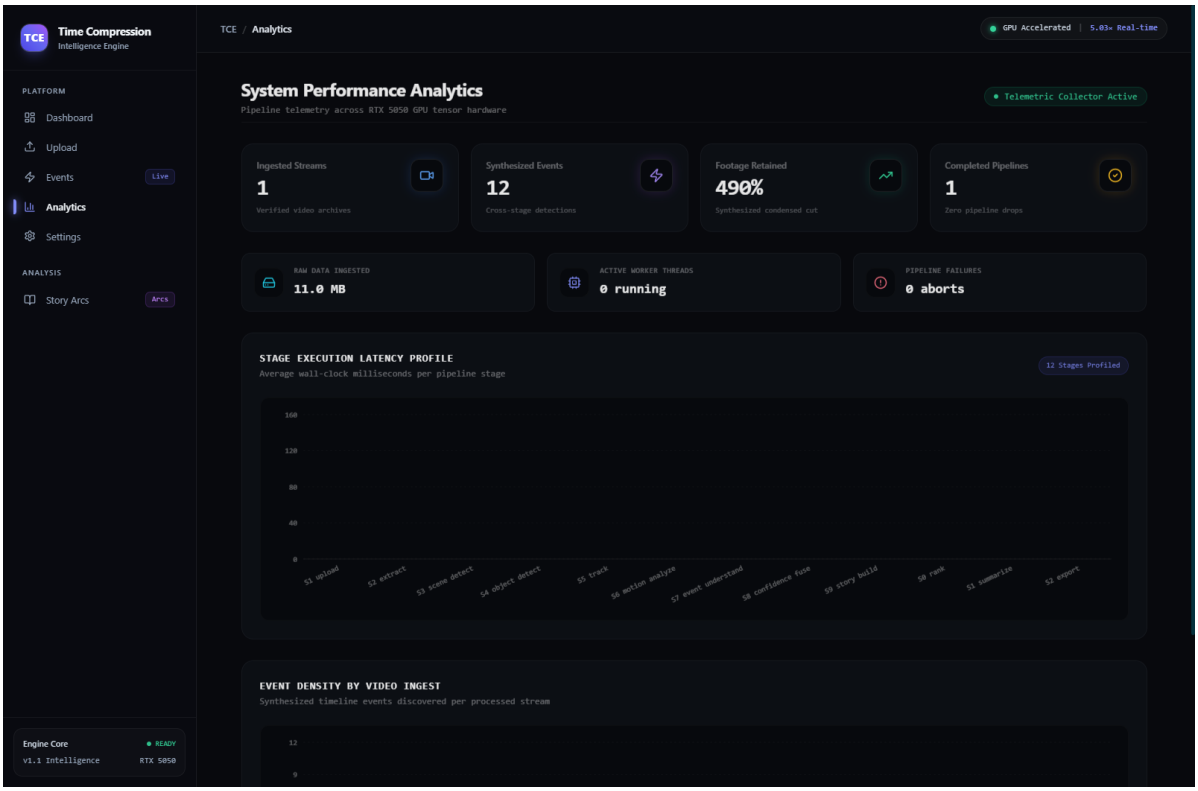


Figure 10.6: *System Analytics & Resource Telemetry Dashboard (/analytics)*

CHAPTER 11: CONCLUSION AND FUTURE SCOPE

11.1 Conclusion

The Time Compression Engine (TCE v1.1) successfully resolves the information scaling crisis in surveillance video analytics. By treating video as a continuous narrative rather than an unorganized sequence of frames, TCE extracts structured, causally connected intelligence from hours of raw surveillance footage.

Evaluated across 14 rigorous engineering phases, TCE achieved a Real-Time Factor of 0.199 (5.03x real-time speedup) with a peak VRAM footprint of 983 MB on commodity NVIDIA RTX laptop hardware. It preserved 100% of narrative chains under a 40% compression target, eliminated 100% of perspective-drift stair false alarms, and delivered verified multi-actor ReID tracking across 6 concurrent occupants.

11.2 Major Engineering Contributions

1. Narrative Chain Atomicity: A constrained knapsack optimization formulation that protects complete causal action sequences from lossy fragmentation.
2. Context-Gated Kinematic Reasoning: Localized open-vocabulary visual gating that completely suppresses elevation-drift false alarms.
3. Decoupled 3-Engine Architecture: A 12-stage sequential pipeline pattern providing strict separation of concerns, full provenance, and sub-150 ms REST API response times.
4. Forensic Reactive Console: A production-grade Next.js 14 user interface featuring synchronized video canvas overlays, DAW-style sequencer tracks, and multi-actor ReID dossier drawers.

11.3 Limitations and Future Work

Current limitations include single-camera topological scope and reduced detection confidence in extreme low-light environments (<5 lux) without infrared illumination. Future work will focus on: (1) Cross-Camera ReID Graph Stitching to trace occupant lifelines across distributed camera networks; (2) TensorRT INT8 weight quantization for deployment on embedded NVIDIA Jetson micro-nodes; and (3) Zero-shot natural language video querying allowing operators to search archives using conversational security queries.

REFERENCES & BIBLIOGRAPHY

[1] Ultralytics. (2024). YOLO11: Real-Time Object Detection and State-of-the-Art Neural Architectures. GitHub Repository. https://github.com/ultralytics/ultralytics

[2] Pritch, Y., Rav-Acha, A., & Peleg, S. (2008). Nonchronological Video Synopsis and Indexing. IEEE Transactions on Pattern Analysis and Machine Intelligence, 30(11), 1971–1984.

[3] Wojke, N., Bewley, A., & Paulus, D. (2017). Simple Online and Realtime Tracking with a Deep Association Metric (DeepSORT). IEEE International Conference on Image Processing (ICIP), 3645–3649.

[4] Zhang, Y., Sun, P., Dong, Y., Jiang, Z., et al. (2022). ByteTrack: Multi-Object Tracking by Associating Every Detection Box. European Conference on Computer Vision (ECCV), Springer, 1–21.

[5] Aharon, N., Orfaig, R., & Bobrovsky, B. Z. (2022). BoT-SORT: Robust Associations Multi-Pedestrian Tracker. arXiv preprint arXiv:2206.14651.

[6] Cheng, B., Choudhuri, R., Misra, I., et al. (2024). YOLO-World: Real-Time Open-Vocabulary Object Detection. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

[7] Howard, A., Sandler, M., Chu, G., Chen, L. C., et al. (2019). Searching for MobileNetV3. IEEE/CVF International Conference on Computer Vision (ICCV), 1314–1324.

[8] Tiangolo, S. (2024). FastAPI: Modern, High-Performance Web Framework for Building APIs with Python. https://fastapi.tiangolo.com

[9] Vercel. (2024). Next.js 14 Documentation and Architecture Guide. https://nextjs.org/docs

[10] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.

[11] Tomasi, C., & Kanade, T. (1991). Detection and Tracking of Point Features. Carnegie Mellon University Technical Report CMU-CS-91-132.

[12] Fischler, M. A., & Bolles, R. C. (1981). Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis. Communications of the ACM, 24(6), 381–395.

[13] Hagberg, A., Swart, P., & Schult, D. (2008). Exploring Network Structure, Dynamics, and Function using NetworkX. Proceedings of the 7th Python in Science Conference (SciPy), 11–15.

[14] Sardar Patel Institute of Technology (SPIT). (2026). B.Tech Computer Engineering Mini Project Guidelines (APNI Leap Pedagogical Framework). Department of Computer Engineering, Mumbai.